

Unit 7: Servlets and Java Server pages (8 Hrs.)

PART-2



By: Kabita Dhital
7th sem(CSIT)

Java Server Pages (JSP)

- JavaServer Pages (JSP) support dynamic content on Web pages. Java code can be inserted into HTML pages by using JSP tags.
- Java Server Pages simplify the delivery of dynamic Web content. They enable Web application programmers to create dynamic content by reusing predefined components and by interacting with components using server-side scripting. Custom-tag libraries are a powerful feature of JSP that allows Java developers to hide complex code for database access and other useful services for dynamic Web pages in custom tags. Web sites use these custom tags like any other Web page element to take advantage of the more complex functionality hidden by the tag. Thus, Web-page designers who are not familiar with Java can enhance Web pages with powerful dynamic content and processing capabilities.
- The classes and interfaces that are specific to JavaServer Pages programming are located in packages **javax.servlet.jsp** and **javax.servlet.jsp.tagext**.
- JSP, or JavaServer Pages, is a technology that allows software developers to create dynamic web pages using Java code embedded within HTML or XML. It's similar to other scripting languages like PHP and ASP, but uses Java.
- **JSP technology enables rapid development of web-based applications** that are server- and platform-independent.

How is JSP More Advantageous than Servlets?

- JSP simplifies web development by combining the strengths of Java with the flexibility of HTML. Some advantages of JSP over Servlets are listed below:
- JSP code is easier to manage than Servlets as it separates UI and business logic.
- JSP minimizes the amount of code required for web applications.
- Generate content dynamically in response to user interactions.
- It provides access to the complete range of Java APIs for robust application development.
- JSP is suitable for applications with growing user bases.

Servlet	JSP
Servlets run faster than JSP.	JSP runs slower than servlet as it takes time to compile the program and convert into servlets.
It is hard to write code in servlet.	It's easier to code in JSP compared to servlets.
In MVC architecture, servlet works as a controller.	In MVC architecture, JSP works as a view for displaying output.
It should be use when there is more data processing involved.	JSP is generally used when there is no involvement of much data processing.
There is no custom tag writing facility in servlets.	You can easily build custom tags that can directly call Java beans.
Servlet is a java code.	JSP is a HTML-based code.
It can accept all protocol requests, including HTTP.	It can only accept HTTP requests.
You can override the <u>service()</u> method.	In JSP, you can't override the <u>service()</u> method.
In Servlet, by default, session management is not enabled, user has to enable it explicitly.	In JSP, session management is automatically enabled.

Example: Servlet with HTML Embedded

- `import jakarta.servlet.ServletException;`
- `import jakarta.servlet.annotation.WebServlet;`
- `import jakarta.servlet.http.*;`
- `import java.io.IOException;`
- `import java.io.PrintWriter;`
- `@WebServlet("/helloServlet")`
- `public class HelloServlet extends HttpServlet {`
- `@Override`
- `protected void doGet(HttpServletRequest request, HttpServletResponse response)`
- `throws ServletException, IOException {`
- `response.setContentType("text/html");`
- `PrintWriter out = response.getWriter();`
- `// HTML content starts here`
- `out.println("<!DOCTYPE html>");`
- `out.println("<html>");`
- `out.println("<head>");`
- `out.println("<title>Servlet HTML Example</title>");`
- `out.println("</head>");`
- `out.println("<body>");`
- `out.println("<h1>Welcome to My Servlet Page</h1>");`
- `out.println("<p>This is an HTML page served from a servlet.</p>");`
- `out.println("</body>");`
- `out.println("</html>");`
- `}`
- `}`

Servlet + JSP (Best Practice)

- `form.jsp`
- `<!DOCTYPE html>`
- `<html>`
- `<head><title>Login Page</title></head>`
- `<body>`
- `<h2>Login Form</h2>`
- `<form action="loginServlet" method="post">`
- `Email: <input type="text" name="email"><br>`
- `Password: <input type="password"`
`name="password"><br>`
- `<input type="submit" value="Login">`
- `</form>`
- `</body>`
- `</html>`

Servlet + JSP (Best Practice)

- **LoginServlet.java – Logic (Controller)**

- `import jakarta.servlet.ServletException;`
- `import jakarta.servlet.annotation.WebServlet;`
- `import jakarta.servlet.http.*;`
- `import java.io.IOException;`
- `@WebServlet("/loginServlet")`
- `public class LoginServlet extends HttpServlet {`
- `@Override`
- `protected void doPost(HttpServletRequest request, HttpServletResponse response)`
- `throws ServletException, IOException {`
- `String email = request.getParameter("email");`
- `String password = request.getParameter("password");`
- `if ("admin@gmail.com".equals(email) && "1234".equals(password)) {`
- `request.setAttribute("userEmail", email);`
- `request.getRequestDispatcher("welcome.jsp").forward(request, response);`
- `} else {`
- `response.sendRedirect("form.jsp?error=Invalid credentials");`
- `}`
- `}`
- `}`

Servlet + JSP (Best Practice)

- **welcome.jsp – Output Page (View)**
- `<%@ page contentType="text/html; charset=UTF-8" language="java" %>`
- `<%`
- `String email = (String) request.getAttribute("userEmail");`
- `%>`
- `<!DOCTYPE html>`
- `<html>`
- `<head><title>Welcome</title></head>`
- `<body>`
- `<h2>Welcome, <%= email %>!</h2>`
- `<p>You have logged in successfully.</p>`
- `</body>`
- `</html>`

Java Server Pages Overview

- There are four key components to **JSPs-directives, actions, scripting elements** and **tag libraries**. **Directives** are messages to the JSP container-the server component that executes JSPs-that enable the programmer to specify page settings, to include content from other resources and to specify custom tag libraries for use in a JSP. **Actions** encapsulate functionality in predefined tags that programmers can embed in a JSP. Actions often are performed based on the information sent to the server as part of a particular client request. They also can create Java objects for use in JSP scriptlets. **Scripting elements** enable programmers to insert Java code that interacts with components in a JSP (and possibly other Web application components) to perform request processing. **Scriptlets**, one kind of scripting element, contain code fragments that describe the action to be performed in response to a user request. **Tag** libraries are part of the tag extension mechanism that enables programmers to create custom tags. Such tags enable Web page designers to manipulate JSP content without prior Java knowledge.
- In some ways, JavaServer Pages look like standard XHTML or XML documents. In fact, JSPs normally include XHTML or XML markup. Such markup is known as fixed-template data or fixed-template text.

- Fixed-template data often helps a programmer decide whether to use a servlet or a JSP. Programmers tend to use JSPs when most of the content sent to the client is fixed-template data and little or none of the content is generated dynamically with Java code. Programmers typically use servlets when only a small portion of the content sent to the client is fixed-template data. In fact, some servlets do not produce content. Rather, they perform a task on behalf of the client, then invoke other servlets or JSPs to provide a response. Note that in most cases servlet and JSP technologies are interchangeable. As with servlets, JSPs normally execute as part of a Web server.
- When a JSP-enabled server receives the first request for a JSP, the JSP container translates the JSP into a Java servlet that handles the current request and future requests to the JSP. Literal text in a JSP becomes string literals in the servlet that represents the translated JSP. Any errors that occur in compiling the new servlet result in translation-time errors. The JSP container places the Java statements that implement the JSP's response in method `_jspService` at translation time. If the new servlet compiles properly, the JSP container invokes method `_jspService` to process the request. The JSP may respond directly or may invoke other Web application components to assist in processing the request. Any errors that occur during request processing are known as request-time errors.
- Overall, the request-response mechanism and the JSP life cycle are the same as those of a servlet. JSPs can override methods `jspInit` and `jspDestroy` (similar to servlet methods `init` and `destroy`), which the JSP container invokes when initializing and terminating a JSP, respectively. JSP programmers can define these methods using JSP declarations--part of the JSP scripting mechanism.

JSP Syntax

1) Declaration: Declare member variables or methods within the generated servlet class.

```
<%! int count = 0; %>    <%! public String getGreeting() { return "Hello!"; } %>
```

2) Scriptlets: Embed blocks of Java code that are executed when the JSP page is requested.

```
<%      String name = "World";      out.println("<h1>Hello, " + name + "!</h1>");  
%>
```

3) Expressions: Evaluate a single Java expression and display its result directly in the output.

```
<%= new Date() %>
```

4) Comments: JSP comments are ignored during translation and do not appear in the generated HTML.

```
<%-- This is a JSP comment --%>
```

5) Expression Language (EL): Provide a simpler way to access data stored in JavaBeans, implicit objects (like request, session, param), and other scopes.

`${user.username}`

`${param.id}`

Example

```
//test.jsp
<html>
  <head>
    <meta http-equiv = "refresh" content = "60" />
    <title>A Simple JSP Example</title>
    <style type = "text/css">
      .big { font-family: helvetica, arial, sans-serif;

        font-weight: bold;
        font-size: 2em; }
    </style>
  </head>
  <body>
    <p class = "big">Simple JSP Example</p>
    <table style = "border: 6px outset;">
      <tr>
        <td style = "background-color: black;">
          <p class = "big" style = "color: cyan;">
            <!-- JSP expression to insert date/time -->
            <%= new java.util.Date() %>
          </p>
        </td>
      </tr>
    </table>
  </body>
</html>
```

Print “Hello Class” 10 times in JSP.

```
<%  
    String hello = "Hello Class";  
    for (int i = 0; i < 10; i++) {  
        out.print("<P>" + hello + "</p>");  
    }  
%>
```

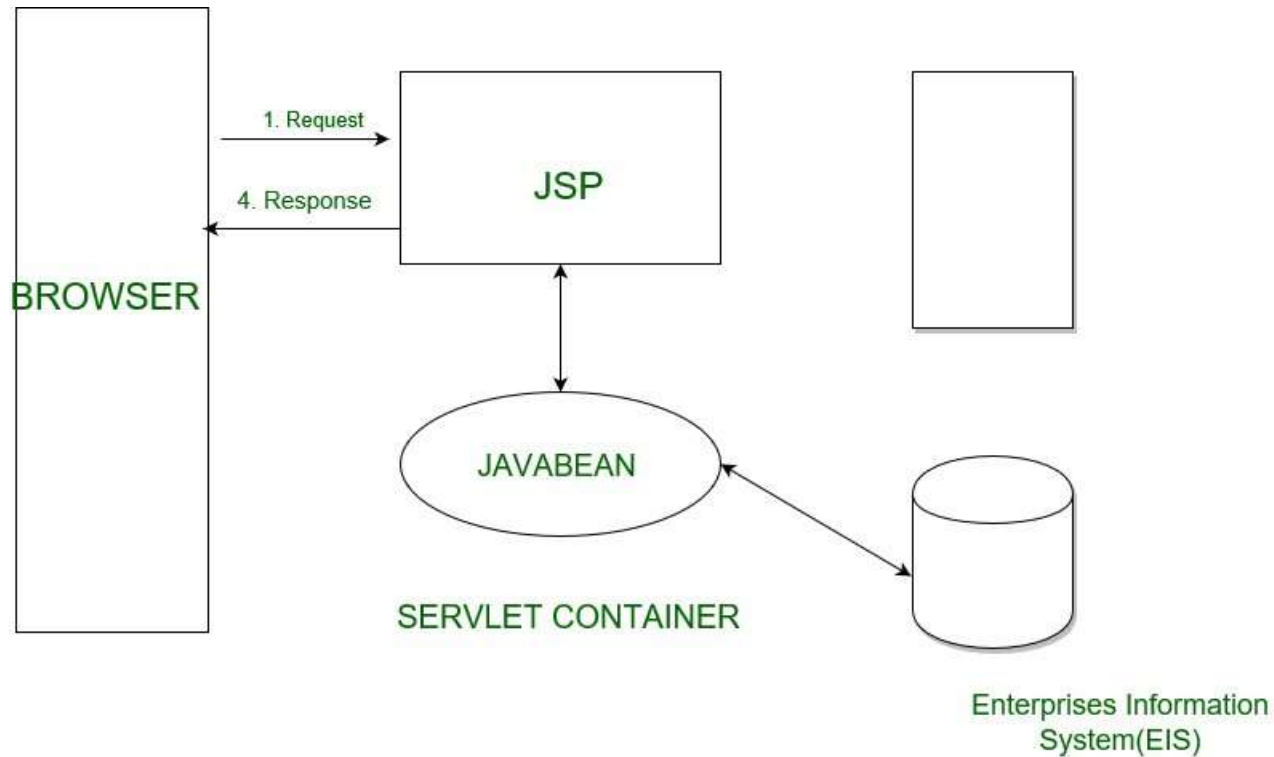
JSP Access Model

- The JSP access model primarily refers to how JavaServer Pages interact with and access data, particularly within the context of web application architectures. **There are two JSP Model Architecture:**
- **JSP Model 1 Architecture:** JSP Model1 Architecture or JSP Centric architecture contains JavaBeans or EJB Model Object, View JSP pages and Action JSP Pages. In model 1 architecture, the incoming request is directly sent to the JSP page from a web browser and JSP page is responsible for processing it and sending back to the client. All the data access is performed using beans so there is still a separation of presentation from content. **Advantages of JSP Centric Model:** Small number of components are required to build the applications.
- A small number of technologies, reducing the learning curve for inexperienced resources.

Disadvantage of JSP Centric Model: Architecture produces a tightly coupled application with hard-coded page names.

- Access JSP pages are primarily java codes, but cannot be developed, compiled and debugged as easily as a java code.
- Re-use of processing and validation is hampered by its placement in form-specific action JSP pages.

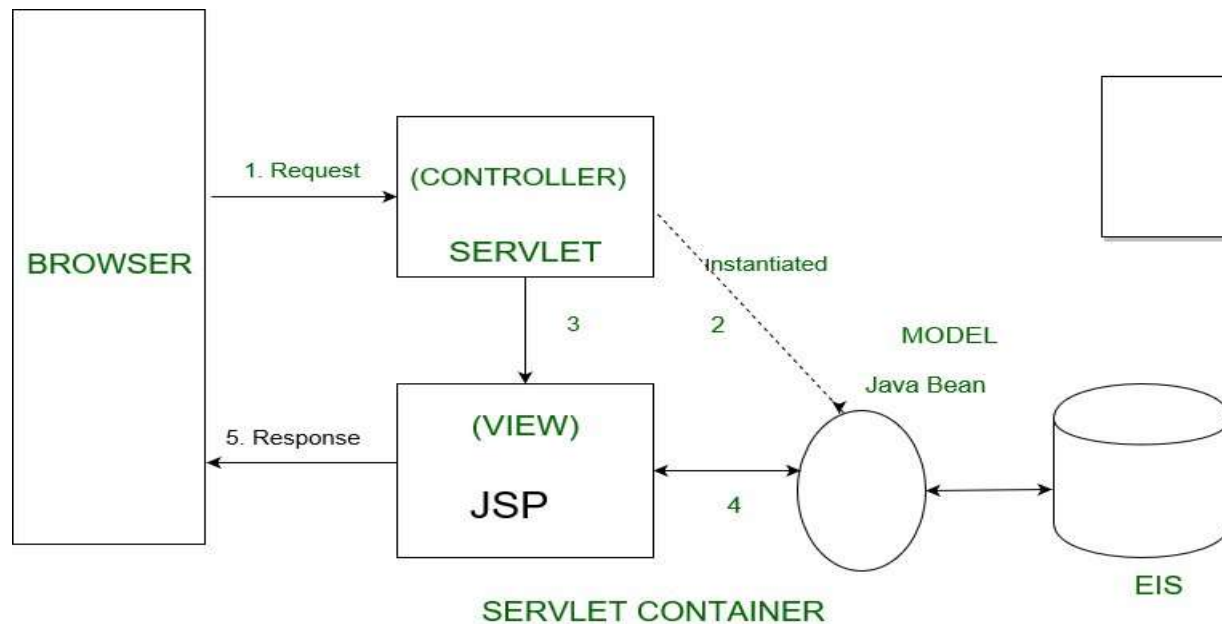
JSP Access Model



Model 1 Architecture

JSP Access Model

- **JSP Model 2 Architecture:** JSP Model 2 Architecture or Servlet-Centric Architecture contains Java beans or EJB Model Objects, view JSP pages and servlet or command classes. It is basically a Model View Controller approach involved notification/event models, direct manipulation of model objects. MVC basically interposes controller component between View and Model Components where the controller is responsible for navigation, presentation-tier logic, validation and emphasizes the separation of presentation logic and model objects.



Model 2 Architecture

- Browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println()` statements and all JSP elements are converted to Java code.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. The output is further passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.
- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.

Form Processing in JSP

JSP handles form data parsing automatically using the following methods depending on the situation –

- **getParameter()** – You call **request.getParameter()** method to get the value of a form parameter.
- **getParameterValues()** – Call this method if the parameter appears more than once and returns multiple values, for example checkbox.
- **getParameterNames()** – Call this method if you want a complete list of all parameters in the current request.
- **getInputStream()** – Call this method to read binary data stream coming from the client.

GET/POST Method Example Using URL

**`http://localhost:8080/main.jsp?
first_name=Corona&last_name
=Virus`**

```
<html>
  <head>
    <title>Using GET Method to Read Form Data</title>
  </head>

  <body>
    <h1>Using GET Method to Read Form Data</h1>
    <ul>
      <li><p><b>First Name:</b>
        <%= request.getParameter("first_name")%>
      </p></li>
      <li><p><b>Last Name:</b>
        <%= request.getParameter("last_name")%>
      </p></li>
    </ul>

  </body>
</html>
```


Exception Handling in JSP

index.jsp

```
<form action="process.jsp">
No1:<input type="text" name="n1" /><br/><br/>
No1:<input type="text" name="n2" /><br/><br/>
<input type="submit" value="divide"/>
</form>
```

error.jsp

```
<%@ page isErrorPage="true" %>

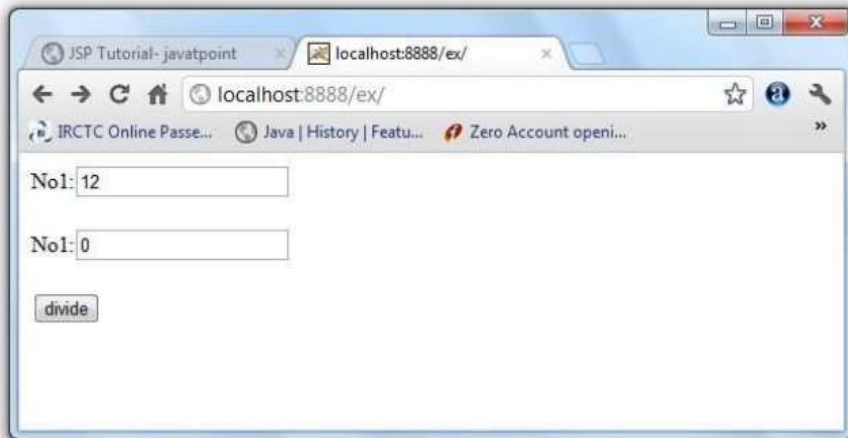
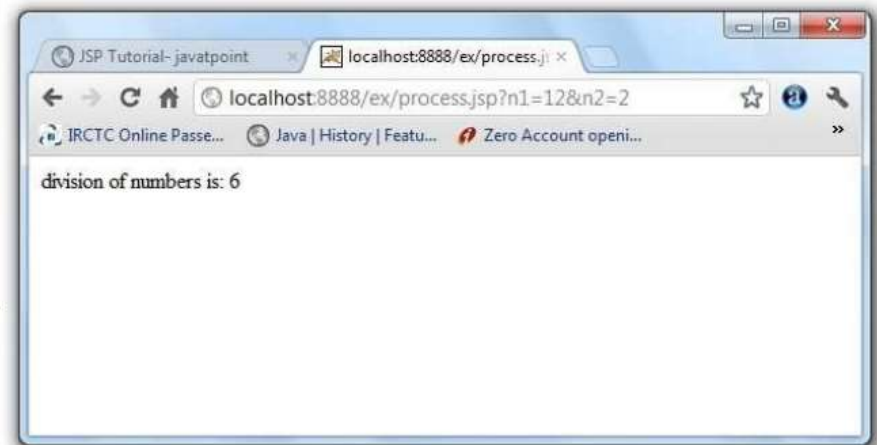
<h3>Sorry an exception occurred!</h3>
Exception is: <%= exception %>
```

process.jsp

```
<%@ page errorPage="error.jsp" %>
<%
String num1=request.getParameter("n1");
String num2=request.getParameter("n2");

int a=Integer.parseInt(num1);
int b=Integer.parseInt(num2);
int c=a/b;
out.print("division of numbers is: "+c);

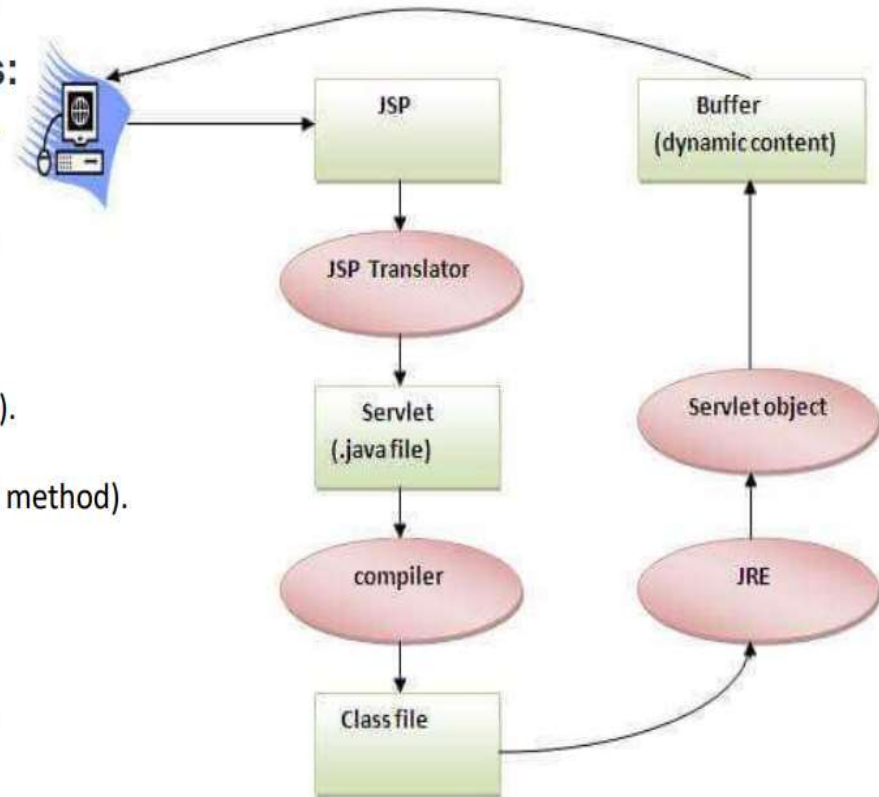
%>
```



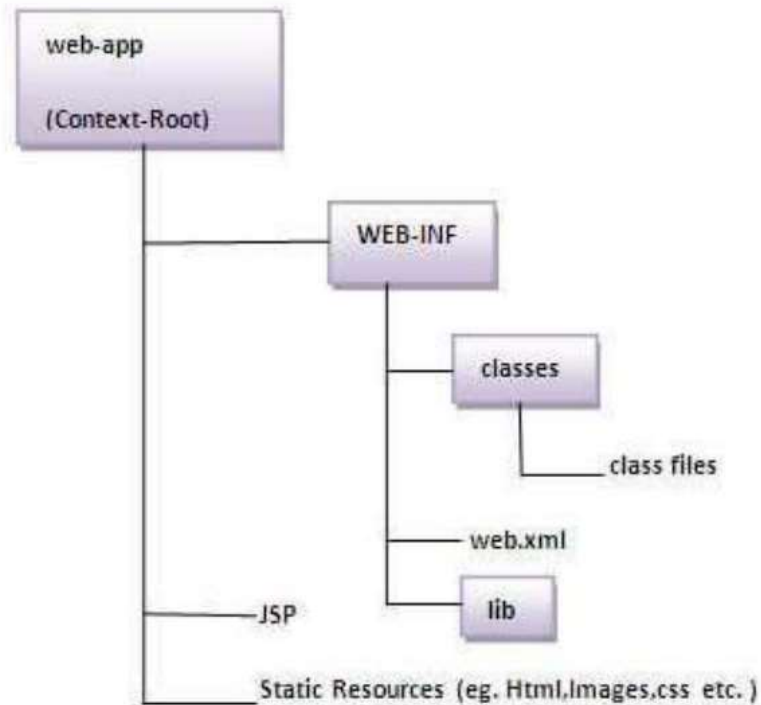
The Lifecycle of a JSP Page

The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page
- Class loading (the classloader loads class file)
- Instantiation (Object of the Generated Servlet is created).
- Initialization (the container invokes `jspInit()` method).
- Request processing (the container invokes `_jspService()` method).
- Destroy (the container invokes `jspDestroy()` method).



The Directory structure of JSP



JSP - Database Access

SELECT Operation

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>
  <head>
    <title>SELECT Operation</title>
  </head>

  <body>
    <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
      url = "jdbc:mysql://localhost/TEST"
      user = "root" password = "pass123"/>

    <sql:query dataSource = "${snapshot}" var = "result">
      SELECT * from Employees;
    </sql:query>
```

```
<table border = "1" width = "100%">
  <tr>
    <th>Emp ID</th>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Age</th>
  </tr>

  <c:forEach var = "row" items = "${result.rows}">
    <tr>
      <td><c:out value = "${row.id}"/></td>
      <td><c:out value = "${row.first}"/></td>
      <td><c:out value = "${row.last}"/></td>
      <td><c:out value = "${row.age}"/></td>
    </tr>
  </c:forEach>
</table>

</body>
/html>
```


INSERT Operation

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>
  <head>
    <title>JINSERT Operation</title>
  </head>

  <body>
    <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
      url = "jdbc:mysql://localhost/TEST"
      user = "root" password = "pass123"/>
    <sql:update dataSource = "${snapshot}" var = "result">
      INSERT INTO Employees VALUES (104, 2, 'Nuha', 'Ali');
    </sql:update>

    <sql:query dataSource = "${snapshot}" var = "result">
      SELECT * from Employees;
    </sql:query>
```

```
<table border = "1" width = "100%">
  <tr>
    <th>Emp ID</th>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Age</th>
  </tr>

  <c:forEach var = "row" items = "${result.rows}">
    <tr>
      <td><c:out value = "${row.id}"/></td>
      <td><c:out value = "${row.first}"/></td>
      <td><c:out value = "${row.last}"/></td>
      <td><c:out value = "${row.age}"/></td>
    </tr>
  </c:forEach>
</table>

</body>
</html>
```

Uploading file to the server using JSP

There are many ways to upload the file to the server. One of the way is by the MultipartRequest class. For using this class you need to have the cos.jar file.

index.jsp

To upload the file to the server, there are two requirements:

1. You must use the post request.
2. encodeType should be multipart/form-data that gives information to the server that you are going to upload the file.

```
<form action="upload.jsp" method="post" enctype="multipart/form-data">  
Select File:<input type="file" name="fname"/> <br/>  
<input type="image" src="MainUpload.png"/>  
</form>
```

upload.jsp

```
<%@ page import="com.oreilly.servlet.MultipartRequest" %>  
<%  
MultipartRequest m = new MultipartRequest(request, "d:/new"  
);  
out.print("successfully uploaded");  
%>
```

DELETE Operation

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>
  <head>
    <title>DELETE Operation</title>
  </head>

  <body>
    <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
      url = "jdbc:mysql://localhost/TEST"
      user = "root" password = "pass123"/>

    <c:set var = "empId" value = "103"/>

    <sql:update dataSource = "${snapshot}" var = "count">
      DELETE FROM Employees WHERE Id = ?
      <sql:param value = "${empId}" />
    </sql:update>

    <sql:query dataSource = "${snapshot}" var = "result">
      SELECT * from Employees;
    </sql:query>
```

```
<table border = "1" width = "100%">
  <tr>
    <th>Emp ID</th>
    <th>First Name</th>
    <th>Last Name</th>
    <th>Age</th>
  </tr>

  <c:forEach var = "row" items = "${result.rows}">
    <tr>
      <td><c:out value = "${row.id}"/></td>
      <td><c:out value = "${row.first}"/></td>
      <td><c:out value = "${row.last}"/></td>
      <td><c:out value = "${row.age}"/></td>
    </tr>
  </c:forEach>
</table>

</body>
</html>
```

Old Questions [TU]

- What is JSP? Differentiate it with servlet. (2+3) [2069]
- Write a simple JSP program to display “Lalitpur, Nepal” 10 times. (5)[2070]
- Write a simple JSP program to display “Tribhuwan University” 10 times. (5) [2071]
- Compare and contrast between Servlets and Java Server pages? How web server responds to servlets and JSP page? Write down sample example. Handling HTTP Request and Response using servlets. [Model Question]

JSP Implicit Objects

- JSP provide access to some implicit object which represent some commonly used objects for servlets that JSP page developers might need to use. For example you can retrieve HTML form parameter data by using **request** variable, which represent the **HttpServletRequest** object.

```
<%  
    String user = request.getParameter("user");  
%>  
  
Hello, <% out.println(user); %>
```

The "request" object is implicit here,
associated with HttpServletRequest object

The "out" object is implicit in JSP, associated with
the JspWriter object.

Types of JSP Implicit Objects

1. out
2. request
3. response
4. config
5. session
6. application
7. page
8. pageContext
9. exception

Types of JSP Implicit Objects

- **1) Request:-** It is inbuilt in `javax.servlet.http.HttpServletRequest`. It has the sole purpose of getting the data entered on a JSP page from the previous JSP page. We often ask users for their details while dealing with JSP forms. This object is used to extract that information and pass it to another JSP page (action page) for validation.

Intro.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>JSP implicit objects</title>
  </head>
  <body>
    <form action="new.jsp">
      Enter your name: <input type="text" name="name" /> <br />

      <input type="submit" value="confirm" />
    </form>
  </body>
</html>
```

new.jsp

```
<%@ page import = " java.util.* " %>
<%
    String name = request.getParameter("name");
    out.print("Hey!!!" + name + " have a good day.");
%>
```

Output

Enter your name:

Types of JSP Implicit Objects

- **2) Response:-** An object from HttpServletResponse is used to modify or handle a response sent back to the client (browser) after processing a request.

intro.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>implicit object</title>
  </head>
  <body>
    <form action="new.jsp">
      OPEN CODING NINJA'S

      <input type="submit" value="OPEN" /><br />
    </form>
  </body>
</html>
```

detail.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="I
SO-8859-1"%>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="ISO-8859-1" />
    <title>JSP response</title>
  </head>
  <body>
    <% response.sendRedirect("https://www.codingninjas.com/"); %>
  </body>
</html>
```

Types of JSP Implicit Objects

- **3) PageContext:-** The javax.servlet.jsp uses this object as an implicit object when accessing the attributes for a page, request, application, or session.

intro.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>implicit object</title>
  </head>
  <body>
    <form action="new.jsp">Enter your name: <input type="text" name="name" /> <input type="submit" value="Enter" /><br /></form>
  </body>
</html>
```

new.jsp

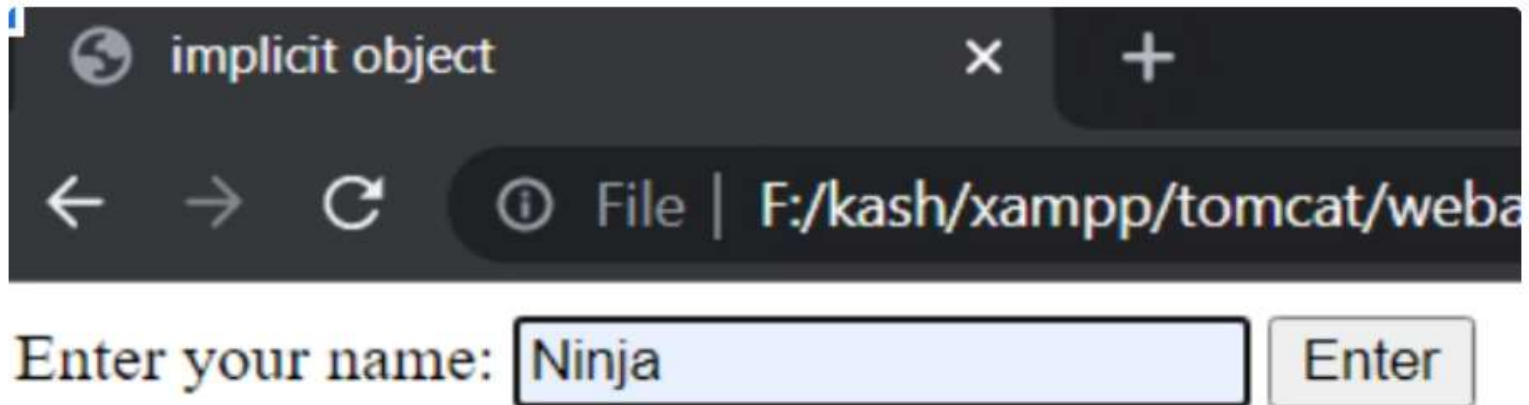
```
<!DOCTYPE html>
<html>
  <head>
    <title>PageContext</title>
  </head>
  <body>
    <% String name = request.getParameter("name"); out.print("Hey!!!!" + name); pageContext.setAttribute("User", name, PageContext.SESSION_SCOPE); %>
    <a href="next.jsp">next jsp page</a>
  </body>
</html>
```

Types of JSP Implicit Objects

next.jsp

```
<!DOCTYPE html>
<html>
  <head>
    <title>page context</title>
  </head>
  <body>
    <% String name = (String) pageContext.getAttribute("User", PageContext.SESSION_SC
OPE); out.print("Hey!!!!" + name); %>
  </body>
</html>
```

Output



The screenshot shows a web browser window with a single tab titled 'implicit object'. The address bar displays the file path 'F:/kash/xampp/tomcat/weba'. Below the address bar, there is a form with the label 'Enter your name:' followed by a text input field containing the word 'Ninja' and a button labeled 'Enter'.

When we load the intro.html file in the browser it will ask for the input after entering the value and pressing the Enter button then new.jsp file will get loaded.

Types of JSP Implicit Objects

4) Out:-It is the most used implicit object that comes from `java.servlet.jsp.JspWriter`. It's work is to write data into a buffer which would be sent to the client as output. It possesses the access to servlet output stream.

Servlet uses `PrintWriter` but `JspWriter` has some additional features to it. `Jsp Writer` has options for buffer and I/O Exceptions which are not available in `PrintWriter`.

Syntax: `out.methodName();`

Out uses various methods such as:

a. void print(): It prints the data to the output. One can specify the data type too.

`out.print(datatype d);`

b. void println(): It prints the data type and terminates the line.

`out.println(datatype d);`

c. void flush(): The contents in the buffer(if present) gets written to the output and the stream is flushed.

`out.flush();`

d. void clear(): It clears the buffer. Even if something is present in the buffer, it doesn't get written to output. It will throw an exception, if this method is called on a buffer that is already flushed.

e. void clear buffer(): It is similar to clear but it doesn't give an exception if called on a flushed buffer.

```
out.clear buffer();
```

f. boolean isAutoFlush(): This method checks that the buffer is set to flush automatically or not.

```
out.isAutoFlush();
```

g. int getBufferSize(): This method returns the size of the buffer in bytes.

```
out.getBufferSize();
```

h. int getRemaining(): This method returns the remaining bytes of buffer that are empty before the buffer overflows.

```
out.getRemaining();
```

```
out.clear();
```

Out2.jsp

```
<html> <head> <title>IMPLICIT OBJECT</title> </head>  
<body>  
<% out.println("first statement"); out.println("second"); out.println("third"); %>  
</body>  
</html>
```

Explanation: Using the method `out.println()`, this example shows the use of `out` implicit object. Output gets shown on the screen.

Output:



Types of JSP Implicit Objects

5) config

It is an object of `javax.servlet.ServletConfig`. It is used to get the information regarding configuration of JSP pages. For example get Servlet Name etc. It gives out the parameters related to servlet mapping initialization.

Syntax: `config.methodName();`

Methods used with config object are:

1. **String getInitParameter(String name)** – This method gets the parameter name.
2. **Enumeration getInitParameterNames()** – This method will return an enumeration of initialization params.
3. **getServletContext()** – It will return reference to Servlet context.
4. **String getServletName()** – It returns the name of the servlet which was defined in the web.xml file used for mapping.

`String s = config.getServletName();`

Types of JSP Implicit Objects

6) Session

session object is the implicit object under `javax.servlet.http.HttpSession`. It is used to get session information as it tracks sessions between client requests. It can be used to get, set as well as remove under scope of session. As session creation and management is a cumbersome project so if you don't want a session to be created we can set the session in page directive as false.

- **Syntax:** `session.getAttribute()`
- Methods used with session object are:
- **a. `setAttribute(String, object)`** – This method will assign a string to the object and save it in session.
- **b. `getAttribute(String name)`** – This method gets the object that is set by `setAttribute`.
- **c. `removeAttribute(String name)`** – Objects that are in the session can be removed using this method.
- **d. `getAttributeNames`** – It returns enumeration of the objects in session.
- **e. `getCreationTime`** – This method returns time when the session was in active mode.

Types of JSP Implicit Objects

- **f. getId** – This method gives out the unique id of the session.
- **g. isNew** – This method checks if a session is new or not. It would return true if cookies are not there.
- **h. getMaxInactiveInterval** – Returns time span, the session was inactive.
- **i. getLastAccessedTime** – This method gives the last accessed time of a session.

7) application

- It is an implicit object of ServletContext from javax.servlet.ServletContext implementation. This object is created by the web container only when an application is deployed. It is generated one per application. As the name suggests, it interacts with servlet.
- It can get set or even remove the attributes under application scope. It can exclusively be used to get the object Request Dispatcher. Using Request Dispatcher, it forwards the request to other pages.
- **Syntax:** application.methodName("value");

Types of JSP Implicit Objects

- Methods available under application scope are:
 - a. Object `getAttribute(String attributeName)`:** This method gets the object available in the attribute and returns it.
`String a = (String)application.getAttribute("Name");`
 - b. void `setAttribute(String attributeName, Object object)`:** This method will set the value of attribute.
`application.setAttribute("Address", "Value of Address")`
 - c. void `removeAttribute(String objectName)`:** This method will remove an attribute from application.
`application.removeAttribute("Name");`
 - d. String `getRealPath(String value)`:** This method will return the absolute path of the file.
`String abspath = application.getRealPath("/form.html");`
 - e. void `log(String message)`:** This method stores logs to default log file of JSP Endine
`application.log(" error 404 page not found");`
 - f. String `getServerInfo()`:** Returns name and version of web container
`application.getServerInfo();`

Types of JSP Implicit Objects

8) Page

- This implicit object comes under java.lang.Object class. Its work is to provide reference to the servlet class generated. Type casting is thus required to access this object as shown below.
- `<% (HttpServletRequest)page.log("message"); %>`
- We use this rarely. Instead, we use it as:
- **Syntax:**
Object page=this;
`<% this.log("message"); %>`

9) Exception

- Exception is an implicit built-in object of java.lang.Throwable. This is useful for handling exceptions in JSP. As these pages handle errors, they can only be used for JSP error pages. This implies that if `<%@ page isErrorPage="true"%>`, then only you can use this object.
- **Syntax is as follows:**
`<%=exception%>`

Object Scope

- In JSP (JavaServer Pages), object scope defines the accessibility and lifespan of objects within a web application.
- There are four distinct scopes available:
- **Page Scope:**
- **Implicit Object:** `pageContext`
- **Lifespan:** An object with page scope is accessible only within the specific JSP page where it was created. Its lifespan is tied to the execution of that single page and ends when the page finishes processing the request.
- **Request Scope:**
- **Implicit Object:** `request`
- **Lifespan:** An object with request scope is accessible across multiple JSP pages that are part of the same HTTP request. This includes pages involved in a `RequestDispatcher.forward()` or `RequestDispatcher.include()` operation. The object's lifespan ends when the HTTP request is completed.
- **Session Scope:**
- **Implicit Object:** `session`
- **Lifespan:** An object with session scope is accessible across all JSP pages within the same HTTP session. This means the object persists as long as the user's session remains active, even if they navigate between different pages in the application. The session ends when it times out or is explicitly invalidated.
- **Application Scope:**
- **Implicit Object:** `application` (representing the `ServletContext`)
- **Lifespan:** An object with application scope is accessible to all JSP pages and servlets within the entire web application. This scope provides a way to share data globally across all users and requests within a single Java Virtual Machine (JVM). Its lifespan is tied to the lifespan of the web application itself, persisting until the application is undeployed or the server is shut down.